

Week 3 – ROS2 Architecture & Communication Model

ROBOTIC EMBEDDED PROGRAMMING
MENG 4060

ANDY ALUBAIDY, PHD, P.ENG.



WE ARE

HUMBER

Review from Last Week



What is ROS?

- **ROS (Robot Operating System) is an open-source framework for developing robot applications.**
- **Provides a collection of tools, libraries, and conventions to simplify building complex and robust robot behavior.**
- **Not an operating system itself, but a middleware that runs on top of Linux.**
- **Enables message-passing between processes ("nodes") in a distributed system.**

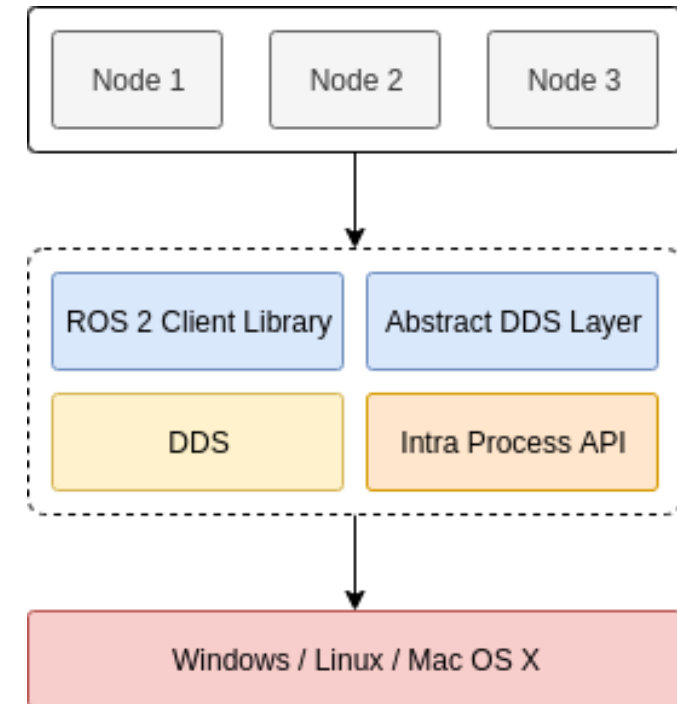
ROS2

Review from Last Week



What is a ROS2 development environment?

- **Distributed System:** Composed of independent nodes communicating via topics, services, or actions.
- **Middleware:** Built on DDS (Data Distribution Service), enabling real-time, scalable communication.
- **Modularity:** Each component of a robot (e.g., perception, planning, actuation) can run as a separate node.



Agenda for today's training session



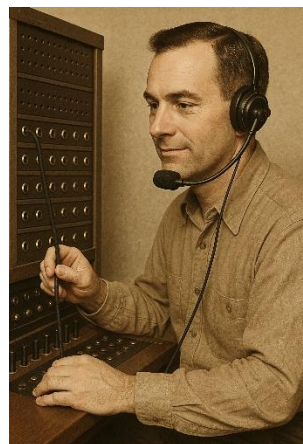
Performance Objective:

- Describe the architecture and core concepts of ROS2 including nodes, publishers/subscribers, services and actions.
- Configure ROS2 on a Linux-based system
- Develop ROS2 nodes in Python or C++ using rclpy or rclcpp
- Manage ROS2 workspaces, packages and files by applying structured software development and modular design practices

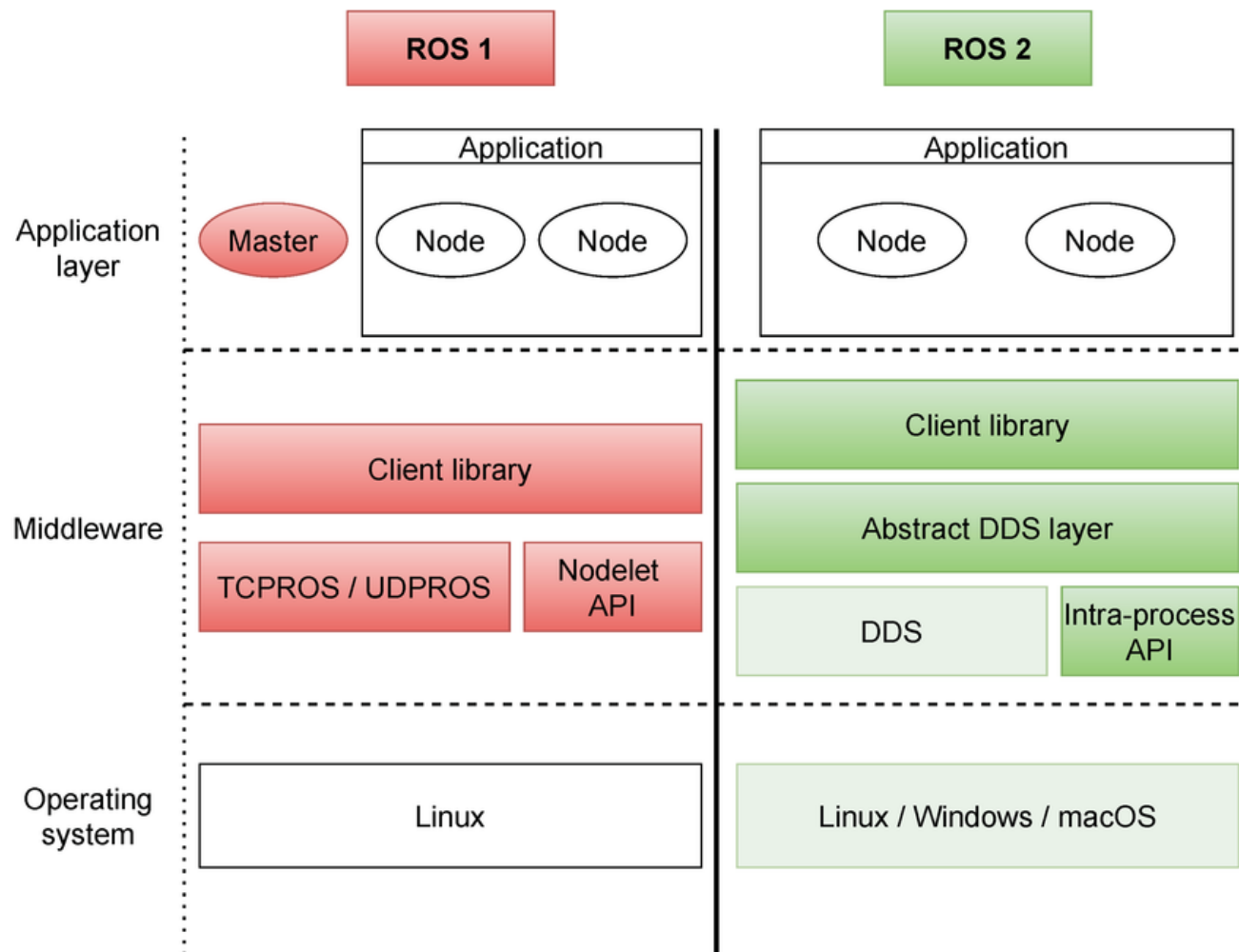
Problem we will be solving today:

- DDS
- Launch Files
- Packages
- System Install (the underlay)
- Workspaces/Overlays

ROS Design: ROS 1 → ROS 2



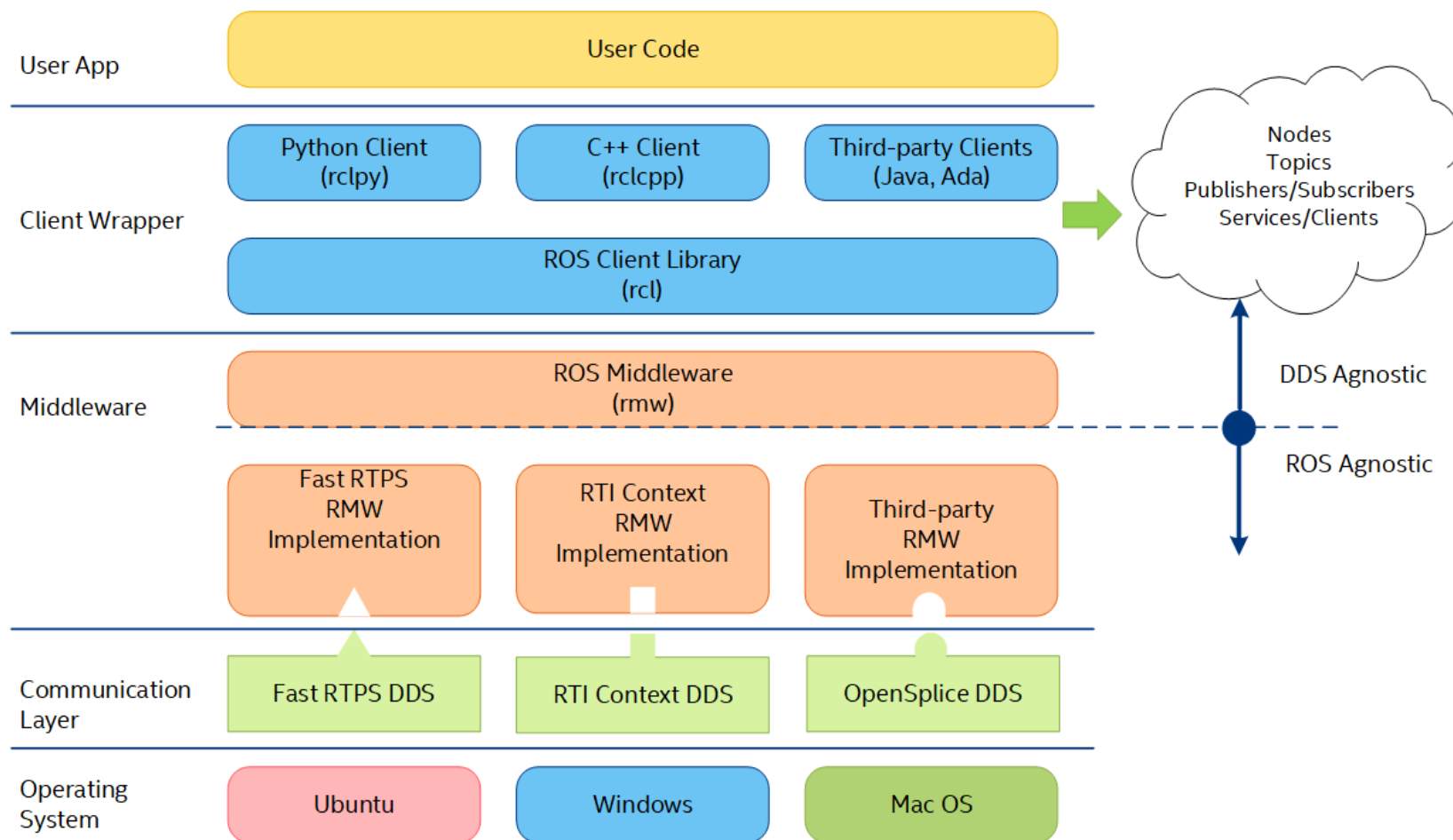
What is the Master's flaw?



DDS: Data Distribution Service

Communication Standard for mission-critical systems

ROS Design- Overview



<https://eci.intel.com/docs/3.3/development/develop/ros.html>

ROS Design – User App Layer



- **This is the code you write as a developer using ROS2.**
- **It includes:**
 - **Nodes (e.g., a robot controller, camera reader, object detector)**
 - **Publishers/subscribers**
 - **Services and actions**
 - **Parameters and logic**

User App

User Code

<https://eci.intel.com/docs/3.3/development/develop/ros.html>

ROS Design – Client Wrapper Layer



- This is the API layer that your application uses to interact with the lower-level system.
- It translates user code into instructions for ROS2.

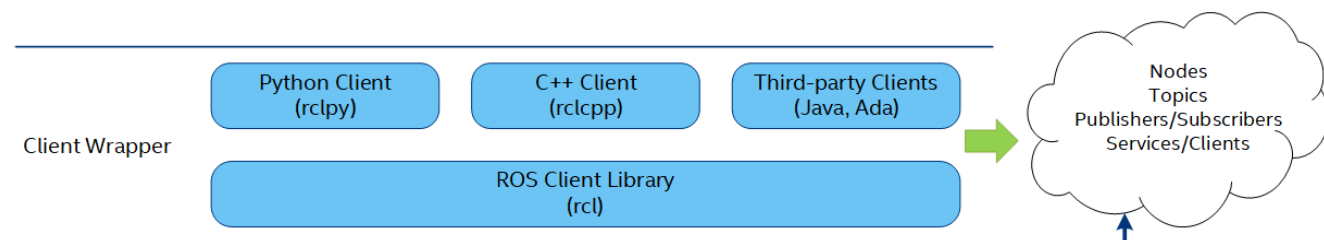
- rclpy → Python client library
- rclcpp → C++ client library
- These libraries wrap the ROS Client Library (RCL) and provide high-level interfaces.

- The Client Wrapper Layer:

- Manages node creation
- Handle publishers/subscribers/services
- Interface with ROS2 parameters, timers, and executors



You don't deal with RCL directly; it's abstracted by rclpy/rclcpp.

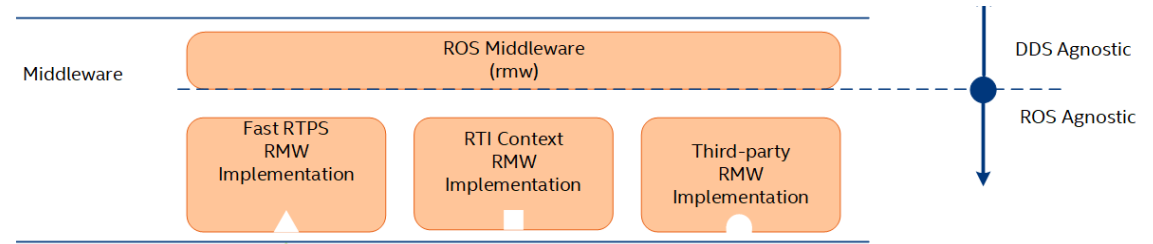


<https://eci.intel.com/docs/3.3/development/develop/ros.html>

ROS Design – Middleware Layer



- **RMW (ROS Middleware)** is a critical abstraction layer in ROS2. It connects the ROS client library to the actual communication middleware (e.g., DDS).
 - Acts like a "middleware plug"
 - Allows ROS2 to support multiple DDS implementations (Fast DDS, Cyclone DDS, RTI Connex, etc.)
 - Ensures that developers can switch middleware without changing application code
-  It's like the adapter between ROS2's internal logic and DDS.

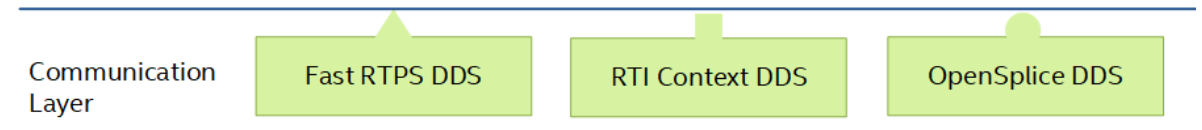


<https://eci.intel.com/docs/3.3/development/develop/ros.html>

ROS Design – Communication Layer



- **This is the core communication engine of ROS2.**
- **Based on the publish-subscribe model**
- **Handles:**
 - **Data discovery (auto-detects publishers/subscribers)**
 - **Message serialization/deserialization**
 - **Transport protocols (UDP, TCP)**
 - **QoS (Quality of Service) like reliability, durability, latency, deadline**
- **DDS is a standardized protocol, so multiple vendors offer compatible implementations.**

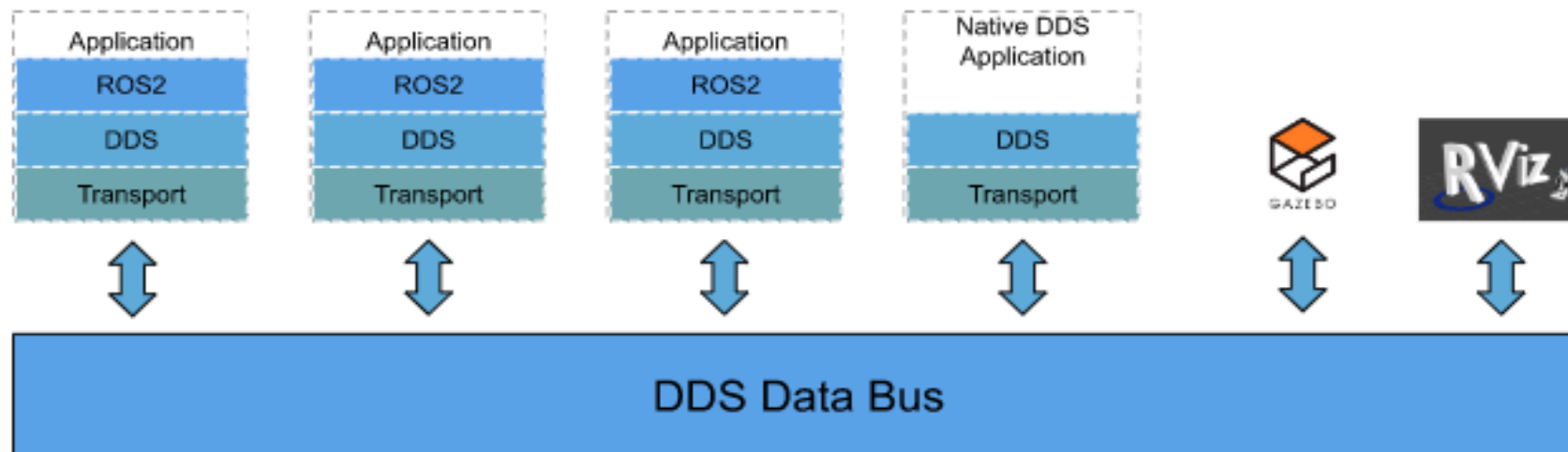


<https://eci.intel.com/docs/3.3/development/develop/ros.html>

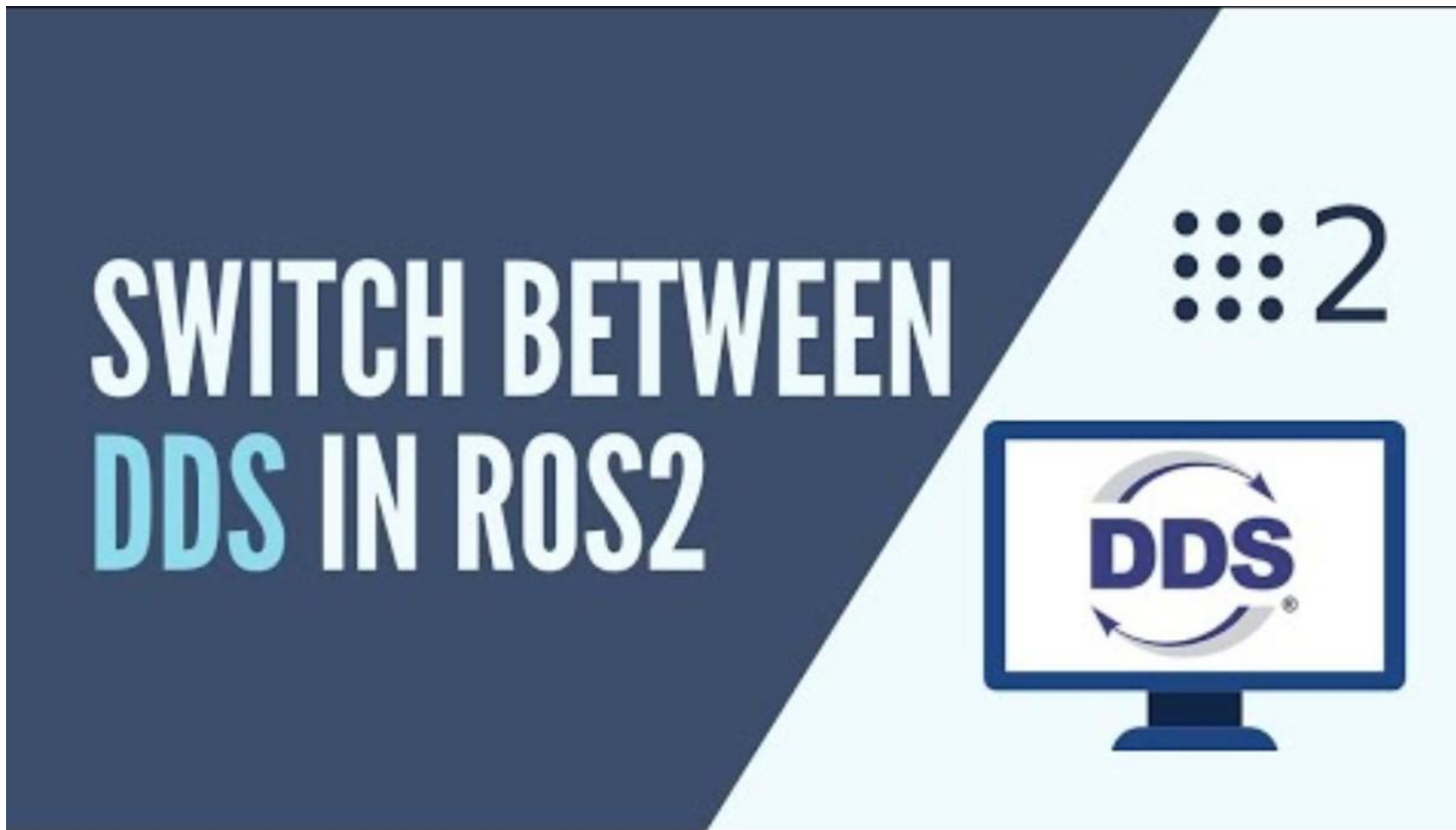
ROS2 Architecture (DDS)



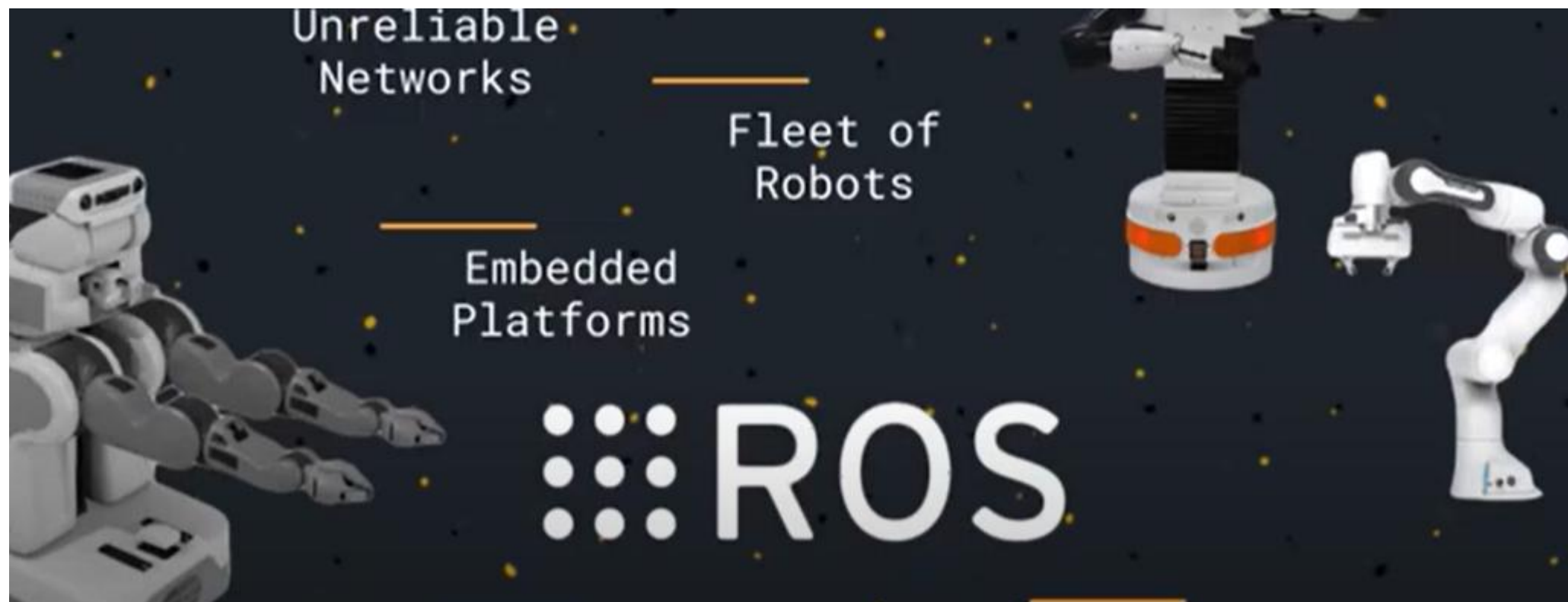
- **DDS is a publish-subscribe protocol standard used by ROS2.**
- **Facilitates real-time, secure, and scalable communication.**
- **Abstracts the network layer for ROS2 nodes.**
- **Enables ROS2 to support features like Quality of Service (QoS), which allows fine control over data delivery.**



How to change DDS



ROS Design - Summary



https://www.youtube.com/watch?v=UQiGZH_KV1g

Namespaces

- **A namespace is a prefix added to node names, topic names, service names, and parameter names.**
- **It allows you to organize and avoid conflicts when running multiple instances of similar nodes.**

Why Use Namespaces?

- Prevent name collisions (e.g., multiple LiDAR drivers both publishing /scan).
- Group related nodes together under a logical scope.
- Make systems modular and scalable.
- Helpful in **multi-robot** systems (e.g., /robot1/camera/image_raw vs /robot2/camera/image_raw).

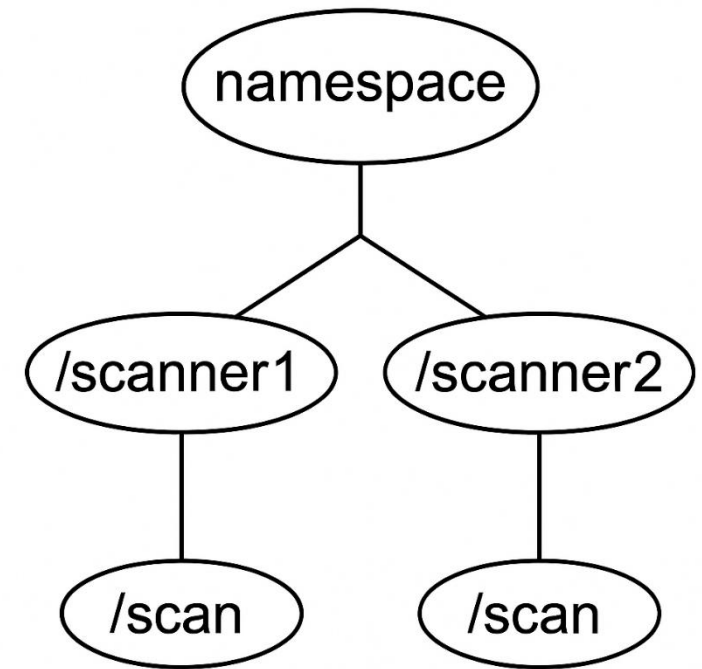
Namespaces - Examples

• Without Namespace `/scan`

```
ros2 run rplidar_ros rplidar_node  
# Publishes to: /scan
```

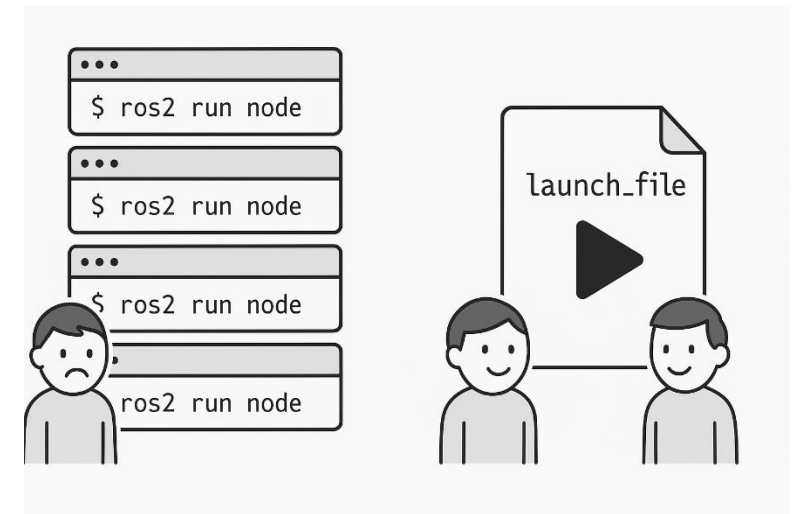
• With Namespace `/scanner1/scan`, `/scanner2/scan`

```
Node(  
    package='rplidar_ros',  
    executable='rplidar_node',  
    namespace='scanner2',  
    parameters=[{'serial_port': '/dev/ttyUSB1'}]  
)  
# Publishes to: /scanner2/scan
```



Launch Files - Motivation

- Running nodes manually is tedious:
 - Multiple terminals
 - Remembering parameters & remappings
 - Stopping everything cleanly
- **Solution → Launch System**
 - Configure & start multiple nodes at once
 - Consistent parameters and remappings
- ROS 1 → XML-based
- ROS 2 Jazzy → Python-based (preferred)



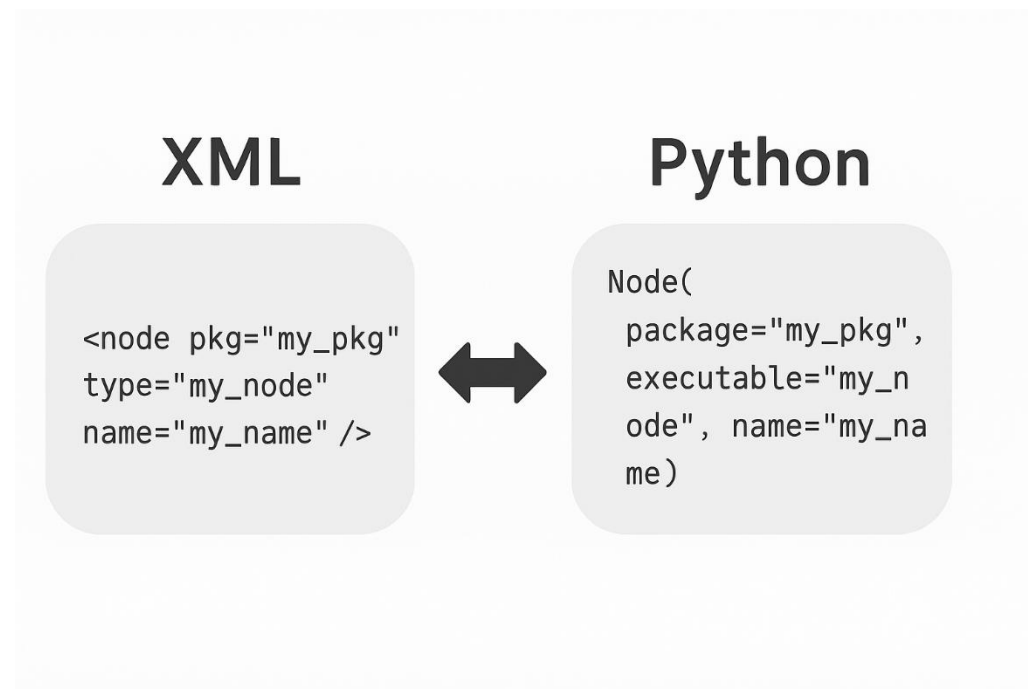
Launch Files in ROS 2 (Python)

- Written as **Python scripts**

- Provide:

- Node definitions
- Parameters
- Remappings
- Namespaces

- **More flexible** than XML (loops, conditions, substitutions, etc.)



Launch File - Example

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='rplidar_ros',
            executable='rplidar_node',
            name='lidar1',
            namespace='scanner1',
            parameters=[{
                'serial_port': '/dev/ttyUSB0'
            }]
        ),
```

- **Node 1**

- Namespace: `/scanner1`
- Topic: `/scanner1/scan`
- Uses `/dev/ttyUSB0`

```
Node(
    package='rplidar_ros',
    executable='rplidar_node',
    name='lidar2',
    namespace='scanner2',
    parameters=[{
        'serial_port': '/dev/ttyUSB1'
    }]
)
])
```

- **Node 2**

- Namespace: `/scanner2`
- Topic: `/scanner2/scan`
- Uses `/dev/ttyUSB1`

Running Launch Files

- Two options:

1. From a package (preferred long-term):

```
ros2 launch <package_name> <launch_file_name>
```

2. Directly from a file (quick test):

```
ros2 launch /path/to/launch/file
```

Running Launch Files

```

PI SSH - robot@robot-pi: ~
robot@robot-pi:~$ ros2 launch testlaunch.launch.py
[INFO] [launch]: All log files can be found below /home/robot/.ros/log/2021-08-03-09-05-41-196191-robot-
pi-22143
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [rplidar_node-1]: process started with pid [22146]
[INFO] [rplidar_node-2]: process started with pid [22148]
[rplidar_node-1] [INFO] [1627945541.644082318] [rplidar_node]: RPLIDAR running on ROS 2 package rplidar_
ros. SDK Version:1.10.0
[rplidar_node-2] [INFO] [1627945541.653055444] [scanner2.rplidar_node]: RPLIDAR running on ROS 2 package
rplidar_ros. SDK Version:1.10.0
[rplidar_node-1] [INFO] [1627945544.167758896] [rplidar_node]: Firmware Ver: 1.28
[rplidar_node-1] [INFO] [1627945544.167922116] [rplidar_node]: Hardware Rev: 5
[rplidar_node-1] [INFO] [1627945544.169430893] [rplidar_node]: RPlidar health status: 0
[rplidar_node-2] [INFO] [1627945544.176640504] [scanner2.rplidar_node]: Firmware Ver: 1.20
[rplidar_node-2] [INFO] [1627945544.176807668] [scanner2.rplidar_node]: Hardware Rev: 0
[rplidar_node-2] [INFO] [1627945544.178368167] [scanner2.rplidar_node]: RPlidar health status: 0
[rplidar_node-2] *WARN* YOU ARE USING DEPRECATED API: checkExpressScanSupported(bool&,_u32), PLEASE MOVE
TO getAllSupportedScanModes()
[rplidar_node-2] *WARN* YOU ARE USING DEPRECATED API: getSampleDuration_uS, PLEASE MOVE TO RplidarScanMo
de::us_per_sample
[rplidar_node-2] [INFO] [1627945544.700928254] [scanner2.rplidar_node]: Current scan mode: Express, max_
distance: 16.0 m, Point number: 2.1K , angle_compensate: 1
[rplidar_node-1] [INFO] [1627945544.713029209] [rplidar_node]: Current scan mode: Express, max_distance:
12.0 m, Point number: 4.0K , angle_compensate: 1

Dev Local - dev@dev-PC: ~
dev@dev-PC:~$ ros2 topic list
/parameter_events
/rosout
/scan_1
/scanner2/scan
dev@dev-PC:~$

```

Why Use Launch Files?

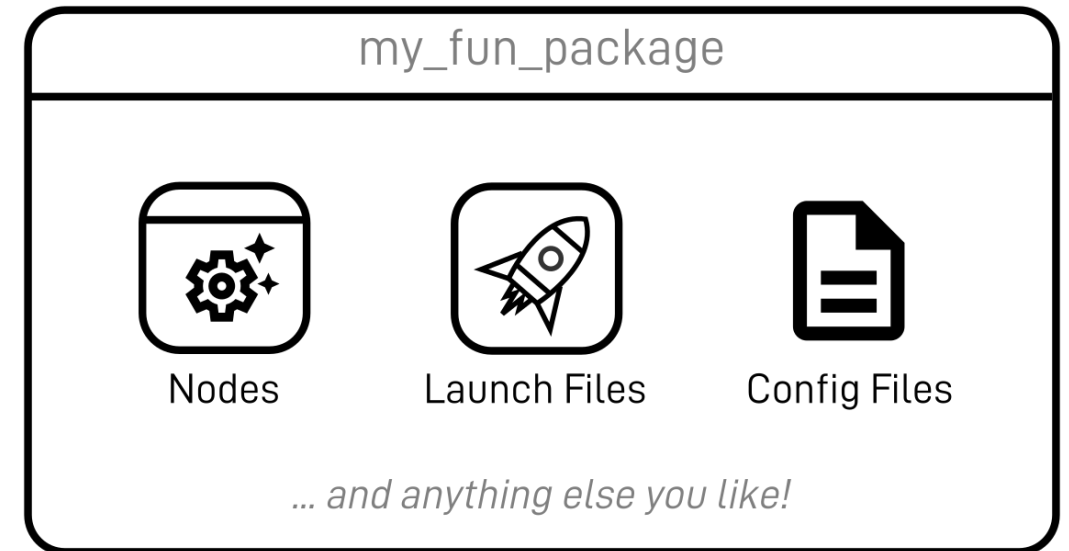
- **Automation:** start many nodes at once
- **Consistency:** avoid manual mistakes
- **Flexibility:** remap topics, namespaces, set params
- **Scalability:** run full robotic systems

Package: Introduction

- ROS systems have **nodes, parameters, launch files** → but how are they organized? **Answer: Packages**
- First level of organization in ROS 2
- Group related files together for:
 - Reuse across projects
 - Modular design

Package: What Can a Package Contain?

- A single node
- A set of nodes
- Configuration files (YAML, JSON, etc.)
- Launch files
- URDF/Xacro robot models
- Software libraries
- ...*anything else useful for robotics development*

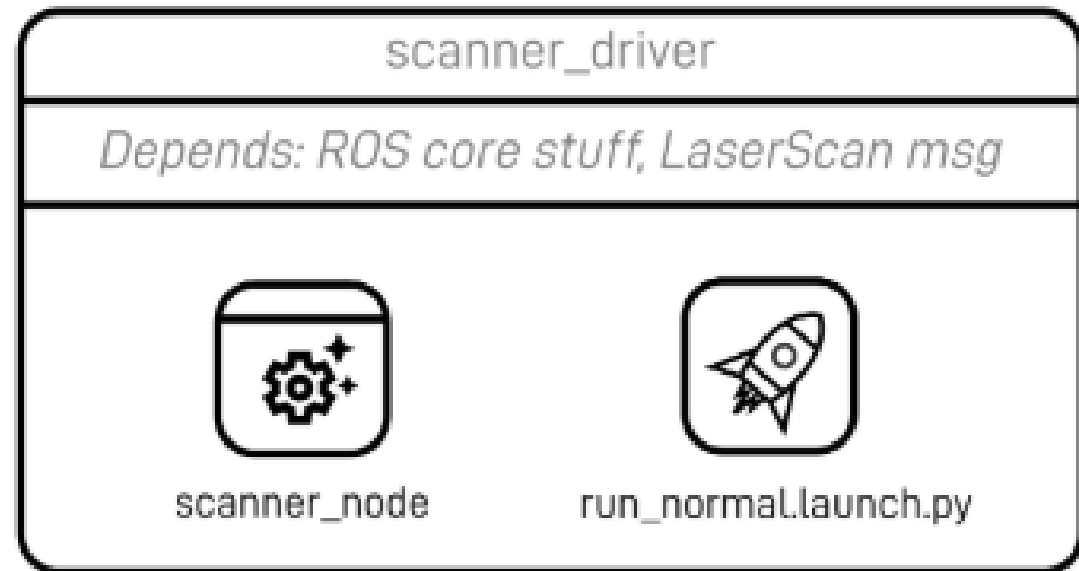


Package: Dependencies

- **Each package includes a list of dependencies**
- **Dependencies = other packages it needs to work**

Package: Example – scanner_driver

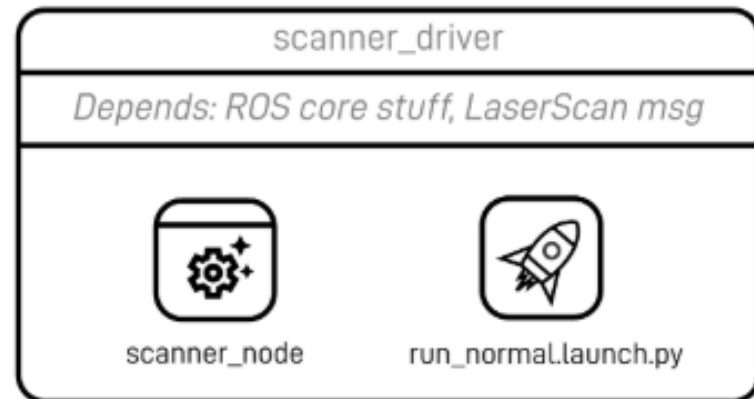
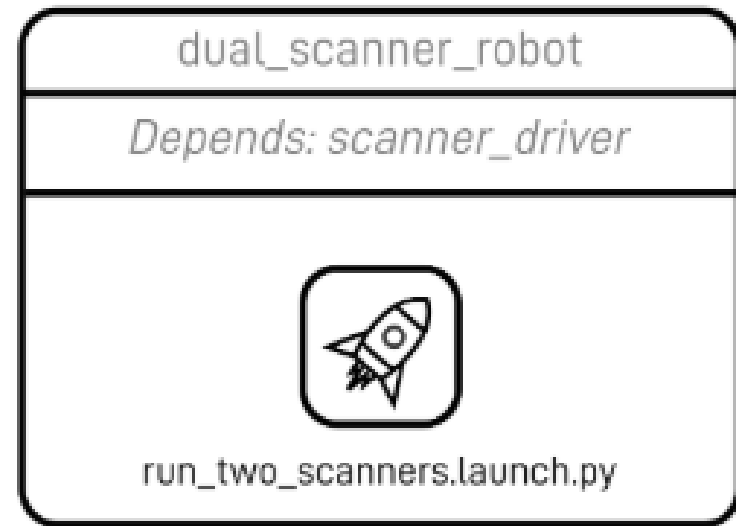
- Contains a node for communicating with the laser
- Includes a launch file that runs the node with some common parameters
- Very few dependencies:
 - Core ROS packages
 - LaserScan messages



This package can be reused in many different projects.

Package: Example – dual_scanner_robot

- Depends on **scanner_driver**
- Contains only a launch file
- Runs two scanner drivers
- Maps them to two separate topics



System Install (the underlay):

- Normally, when we install software (e.g., using apt), **files go into special locations where the system expects them.**
- **Example:** when you type a command in the terminal, it searches through a list of paths to find and run the program.

System Install (the underlay): Multiple Versions

- ROS could use the same approach, but some users want **multiple ROS versions** installed and swappable. This could cause **conflicts**.
- Solutions in other domains:
 - Virtual machines
 - Containers (Docker)
 - Virtual environments (venv, conda)
- **ROS provides its own solution.**

System Install (the underlay): ROS Directory

- Installing a ROS distribution creates a directory in /opt/ros.
- Example: /opt/ros/jazzy
- This directory contains:
 - All core packages
 - Configuration files

System Install (the underlay): setup.bash

- Inside /opt/ros/jazzy is **setup.bash**.
- Contains all information needed to make ROS visible to the system.
- When a terminal first opens, it knows nothing about ROS until you **source** it:

```
source /opt/ros/jazzy/setup.bash
```

System Install (the underlay): Underlay

- This base ROS installation is called the **underlay**.
- Our own packages are built **on top of the underlay**.
- Technically, multiple underlays can exist.
- Typically, “underlay” refers to the main ROS installation.

Workspaces/Overlays: Introduction

- In ROS, we create a **workspace** to keep all our own packages.
- Generally → one workspace per project.
- Contains relevant packages for that project.
- Refers to **packages compiled from source** (written ourselves or pulled from GitHub).
- System-installed packages are **always visible** to all workspaces

Workspaces/Overlays: Why a Workspace?

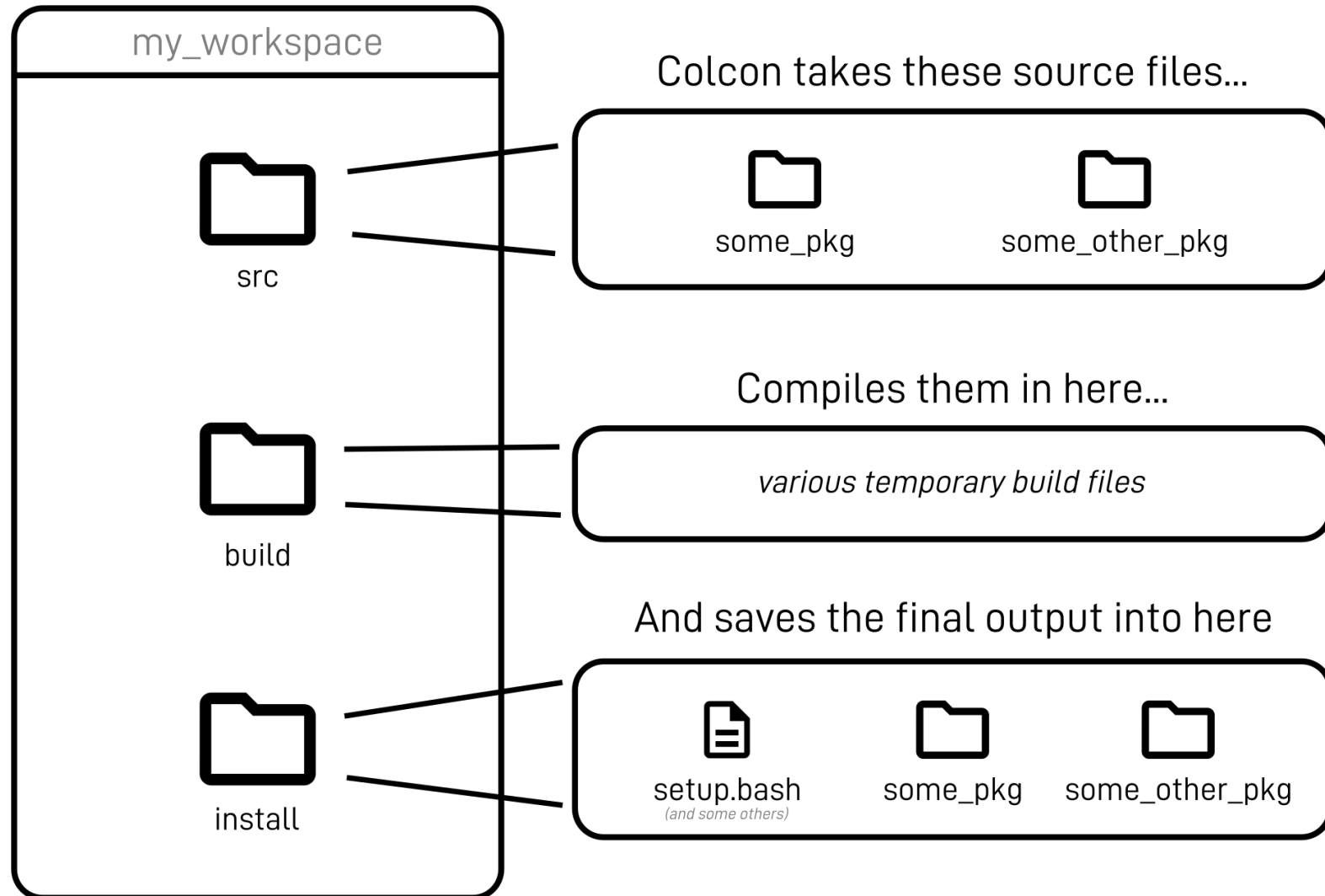
- Normal build tools: don't know where ROS components are.
- ROS also wouldn't know where to find **our packages**.
- Solution → **colcon build tool**.

Workspaces/Overlays: Colcon Build

- colcon **builds the workspace by organizing:**
 - src/ → **source files**
 - build/ → **temporary build files**
 - install/ → **compiled & installed packages**
- Creates **setup.bash** file → lets the system find our packages.
- Must source workspace before use:

```
source /opt/ros/jazzy/setup.bash
```

Workspaces/Overlays: Colcon Build



QoS (Quality of Service)

- **New feature in ROS 2** controlling how messages are transferred.
- Defines **rules** for publishers & subscribers:
 - **Reliable**: every message must arrive (may cause delays).
 - **Best Effort**: latest messages preferred, some may be dropped.
- **Important**: Publisher & subscriber must have **compatible QoS**.
 - If not → **no communication**.
- **Common issue**:
 - Example → Publisher = *Best Effort* vs Subscriber = *Reliable* → messages fail.
 - Changing subscriber to *Best Effort* fixes it.
- **Debugging**:
 - Use `ros2 topic info /topic_name --verbose` **to see QoS settings**.

Talker & Listener Example (ROS 2 Jazzy)

1. Open new terminal

2. Source ROS 2 Environment

```
source /opt/ros/jazzy/setup.bash
```

3. Run the Talker Node (Talker (Publisher): Publishes messages on /chatter → sends messages)

```
ros2 run demo_nodes_cpp talker
```

4. Open another terminal

5. Source ROS 2 Environment

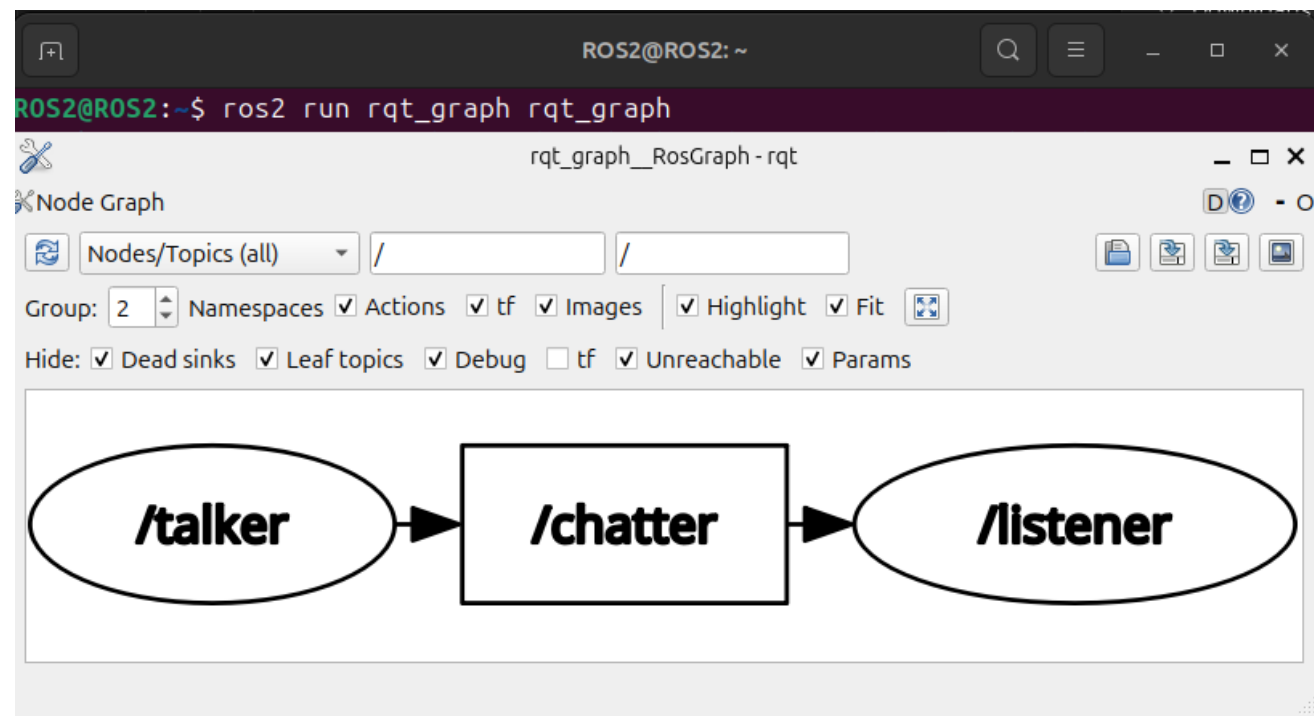
```
source /opt/ros/jazzy/setup.bash
```

**6. Run the Listener Node: Subscribes to /chatter (Topic /chatter → carries messages)
Listener (Subscriber) → receives messages**

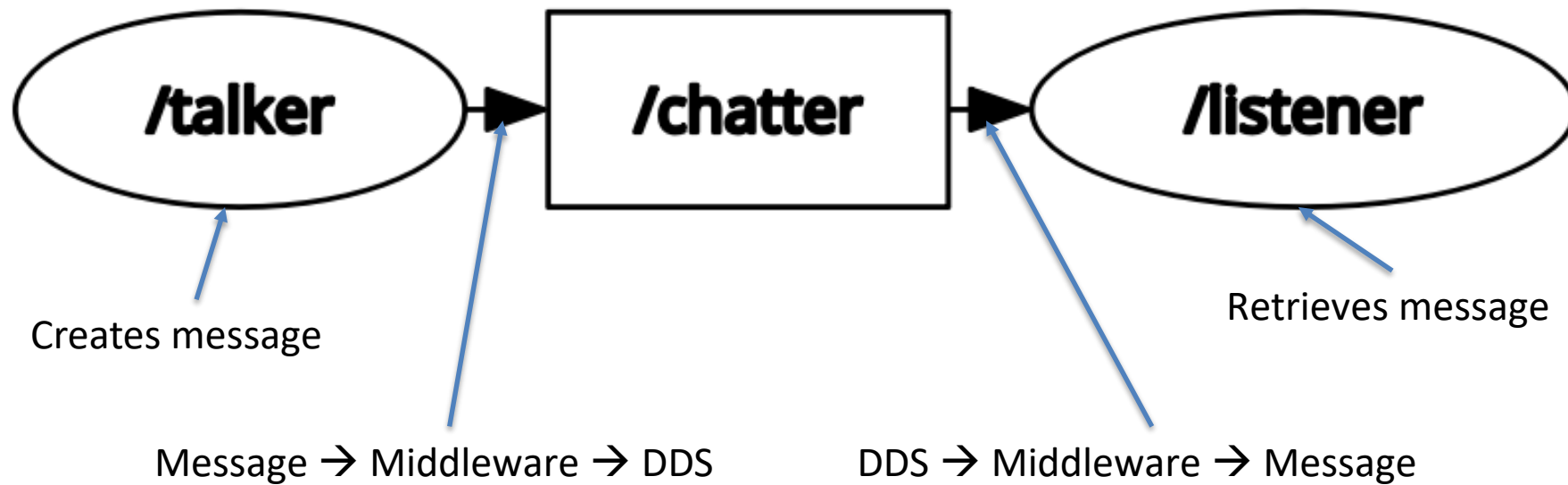
```
38 ros2 run demo_nodes_cpp listener
```

ROS2 – Talker Example

- To see the compute graph type the command in a new terminal
 - `ros2 run rqt_graph rqt_graph`



ROS2 – Talker Example



Key things to remember...



- ***Key tips***
 - ***ROS2 architecture consists of:***
 - **User APP Layer** - code you write as a developer using ROS2
 - **Client Wrapper Layer** - translates user code into instructions for ROS2
 - **Middleware Layer** - connects the ROS client library to the DDS communication
 - **Communication Layer** - core communication engine of ROS2
 - **Operating System** - ROS2 runs on the underlying OS
- ***Practicing outside the classroom***

Read the following articles to better understand IoT and its applications

 - <https://github.com/ros2/ros2cli>
 - https://github.com/ubuntu-robotics/ros2_cheats_sheet/blob/master/cli/cli_cheats_sheet.pdf
 - <https://docs.ros.org/en/kilted/Concepts/Basic/About-Interfaces.html>
 - <https://www.omg.org/omg-dds-portal>
 - <https://github.com/eProsima/Fast-DDS>
 - <https://fast-dds-monitor.readthedocs.io/en/latest/>

Summary



- **DDS** – Core middleware for decentralized communication
- **Launch Files** – Automate running multiple nodes with parameters, remappings, namespaces
- **Packages** – First level of organization; contain nodes, configs, launch files, dependencies
- **System Install (Underlay)** – Base ROS 2 installation at /opt/ros/jazzy **with** setup.bash
- **Workspaces/Overlays** – User-created environments built with colcon; **layered on top of underlay**
- **QoS** – Defines communication reliability & performance; must match between publishers & subscribers
- **Talker–Listener** – Classic ROS 2 demo of pub/sub communication on the /chatter **topic**

THANK YOU.



WE ARE

HUMBER